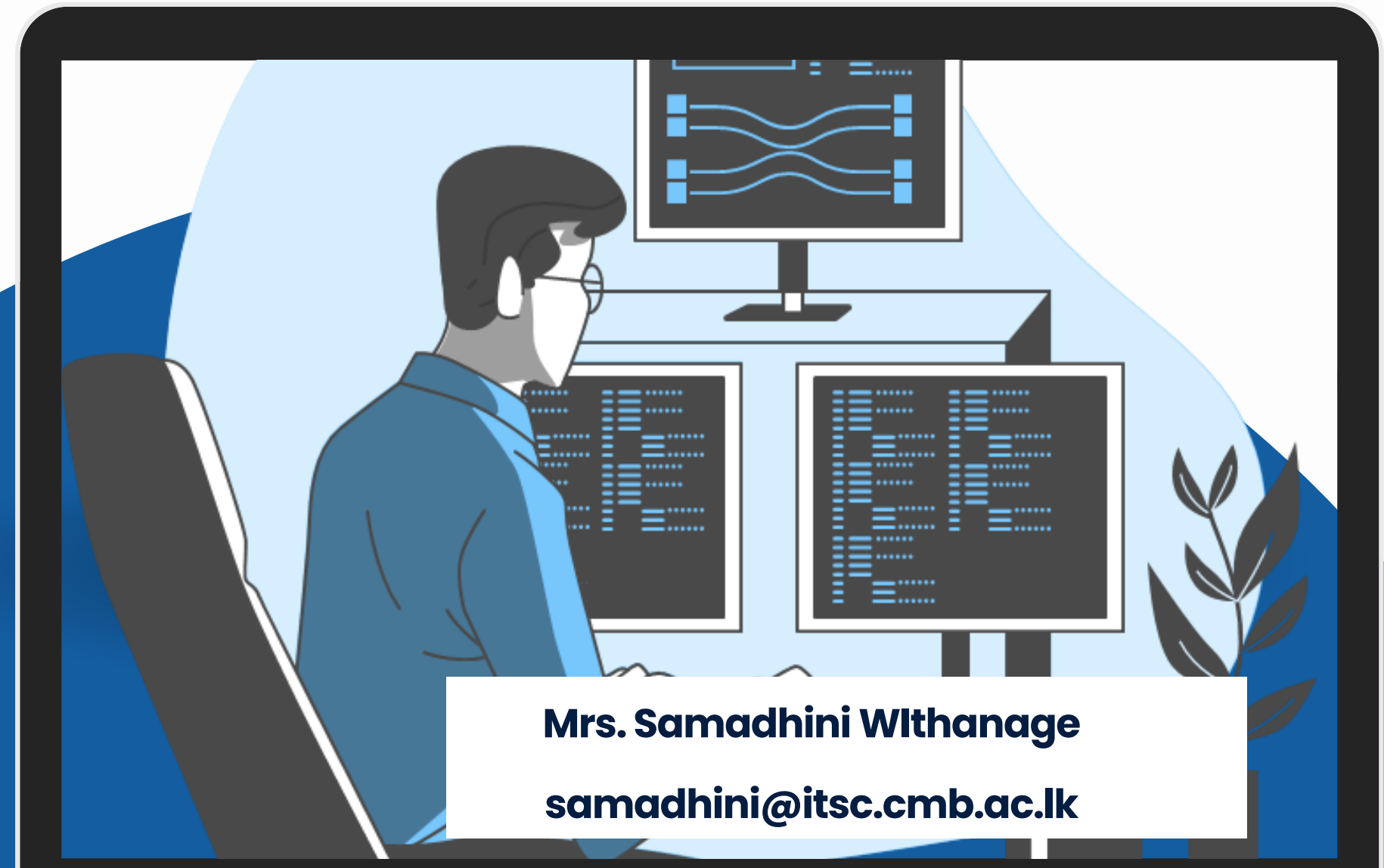


IT 3003

Advanced Programming Techniques



Mrs. Samadhini Wlthanage

samadhini@itsc.cmb.ac.lk

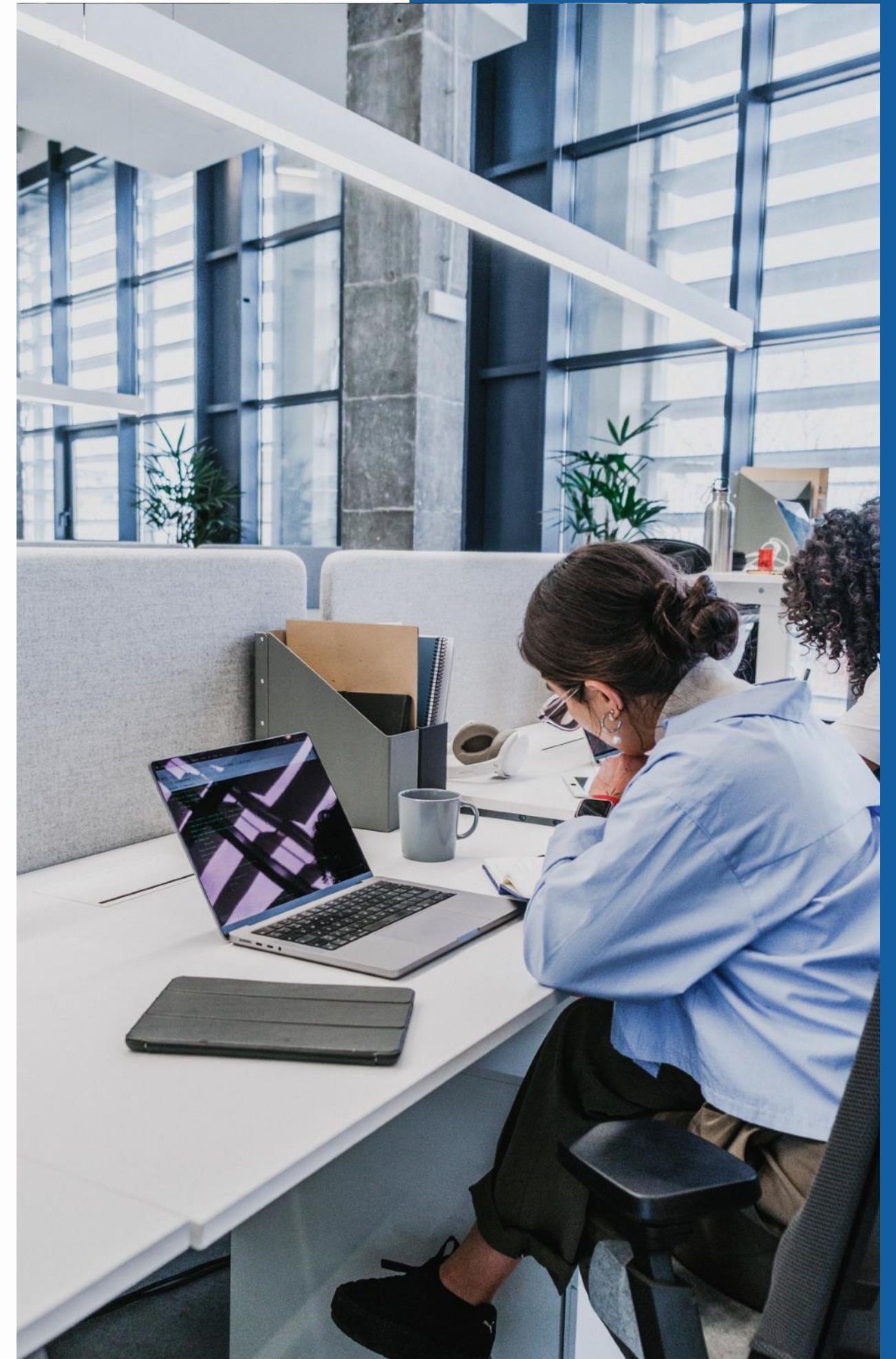
A background image of a modern office environment. In the foreground, a woman with curly hair is sitting at a desk, working on a laptop. To her right, a man in a light blue shirt is also working at a desk. In the background, another person is visible, and there are various office supplies like pens, paper, and a water bottle on the desks. The office has a clean, minimalist design with white walls and desks.

Lesson 5

UML & Object-Oriented Design

Lesson Outline

- ▶ What is UML
- ▶ Importance of UML
- ▶ UML Tools
- ▶ Types of UML
 - ▶ Class Diagram
 - ▶ Use Case Diagram
 - ▶ Activity Diagram
 - ▶ State Diagram



What is UML (Unified Modeling Language)

▶ UML is a standard visual language used to:

- Design software systems
- Represent object-oriented concepts
- Visualize relationships between classes and objects
- Improve communication among developers

Why UML is Important

Advantages of UML

- Simplifies complex systems
- Helps before coding
- Improves software planning
- Reduces design errors
- Enhances teamwork and documentation

UML Tools

UML (Unified Modeling Language) tools are software applications used to create, visualize, and manage UML diagrams for software system design and documentation.

These tools help **developers, analysts, and designers** represent the structure and behavior of a system before implementation.

UML tools improve:

- System planning
- Communication among team members
- Software documentation
- Code organization and design understanding

UML Tools cont..

- ▶ StarUML → Popular for professional UML modeling and code generation.
- ▶ Visual Paradigm → Supports UML, ER diagrams, BPMN, and team collaboration.
- ▶ Lucidchart → Web-based tool for collaborative UML diagram creation.
- ▶ **draw.io** → **Free and simple tool for creating UML diagrams online.**
- ▶ Microsoft Visio → Widely used in business and enterprise environments for professional diagrams.
- ▶ Enterprise Architect → Advanced modeling tool used in large-scale software engineering projects.

UML Tools cont..

▶ **draw.io** - Best Recommended UML Tools for Undergraduate Students.

Why it is best:

- Completely free
- Very simple drag-and-drop interface
- No installation required (web-based)
- Supports all common UML diagrams
- Easy for beginners to learn quickly

Best for:

- Lab practicals
- Assignments
- Group projects
- Quick diagram creation

Recommended as the best starting UML tool for undergraduate practical sessions.

UML Tools cont..

Purpose	Recommended Tool
Beginner UML teaching	diagrams.net
Classroom demonstrations	diagrams.net
Group assignments	Lucidchart
Advanced SE modules	Visual Paradigm
Developer/Code-based UML	PlantUML

Types of UML Diagrams

Structural Diagrams (Represent the static structure of a system)

- Class Diagram
- Object Diagram
- Component Diagram

Behavioral Diagrams (Represent the dynamic behavior of a system)

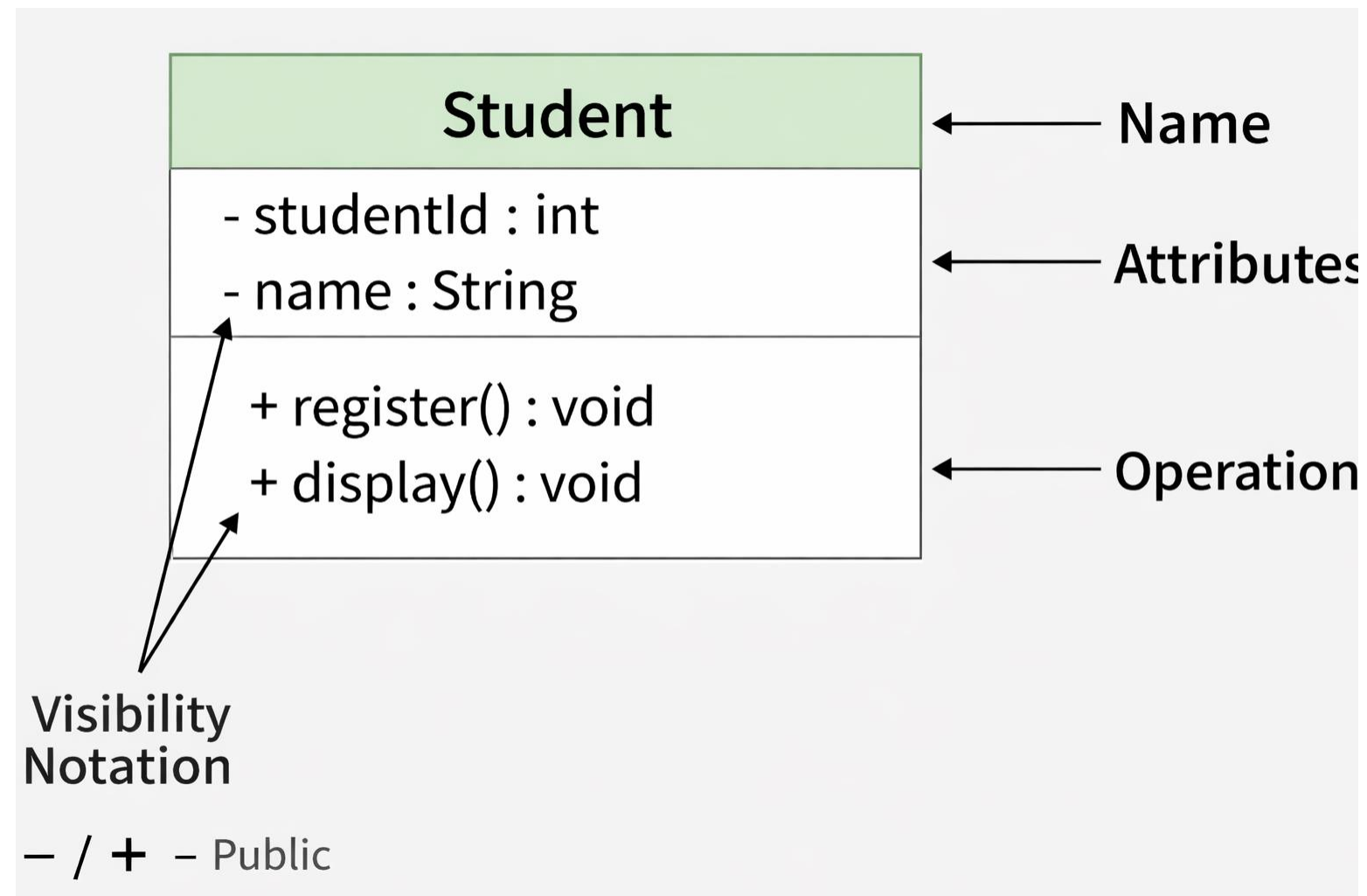
- Use Case Diagram
- Sequence Diagram
- Activity Diagram

Class Diagram

- ▶ A UML class diagram shows the structure of a system by displaying its **classes, their attributes, methods, and the relationships** between them.
- ▶ It helps the team understand how the system is organized and how components interact.
- ▶ Class diagrams are mainly used during:
 - System analysis
 - Software design
 - Database planning
 - Object-oriented programming

Class Diagram cont..

► UML Class Notation



Class Diagram cont..

▶ UML Class Notation Cont..

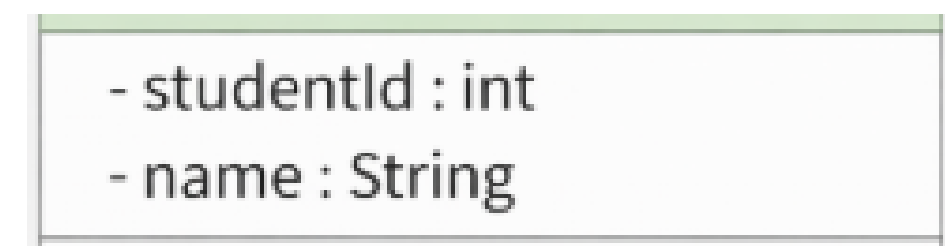
1. Class Name

- The class name is written in the **top compartment** of the class box.
- It is usually **centered and bold**.
- Represents the name of the class.



2. Attributes

- Attributes are also called **properties or fields**.
- They represent the **data members** of the class.
- Written in the **second compartment** of the class box.
- Usually include:
 - Visibility symbol
 - Attribute name
 - Data type

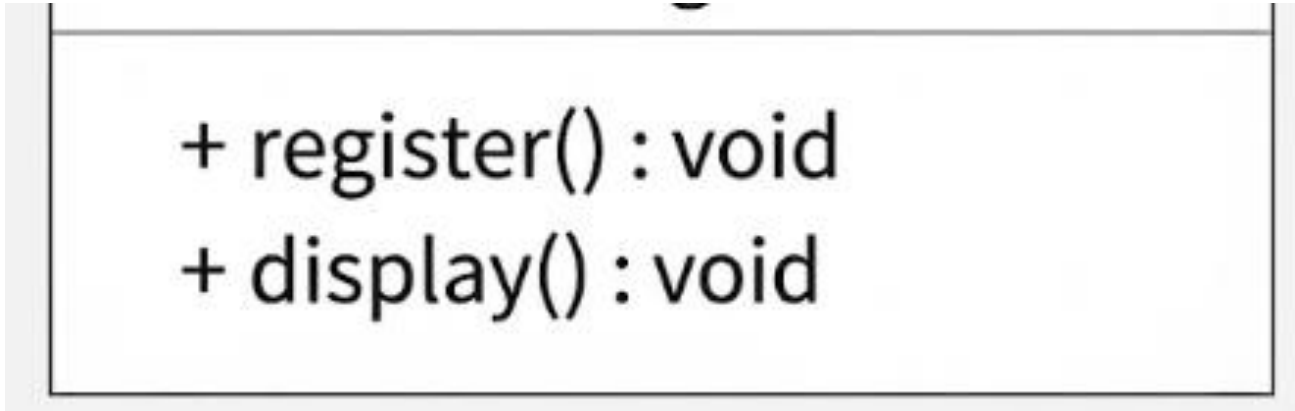


Class Diagram cont..

▶ UML Class Notation Cont..

3. Methods

- Methods are also called **functions or operations**.
- They represent the **behavior of the class**.
- Written in the **third compartment** of the class box.
- Usually include:
 - Visibility symbol
 - Method name
 - Parameters
 - Return type.



```
classDiagram
    class MyClass {
        + register() : void
        + display() : void
    }
```

+ register() : void
+ display() : void

Class Diagram cont..

▶ UML Class Notation Cont..

Visibility Notation

Visibility symbols define the **access level** of attributes and methods.

Symbol	Visibility	Description
+	Public	Accessible from all classes
-	Private	Accessible only within the same class
#	Protected	Accessible within subclasses
~	Package / Default	Accessible within the same package

Class Diagram cont..

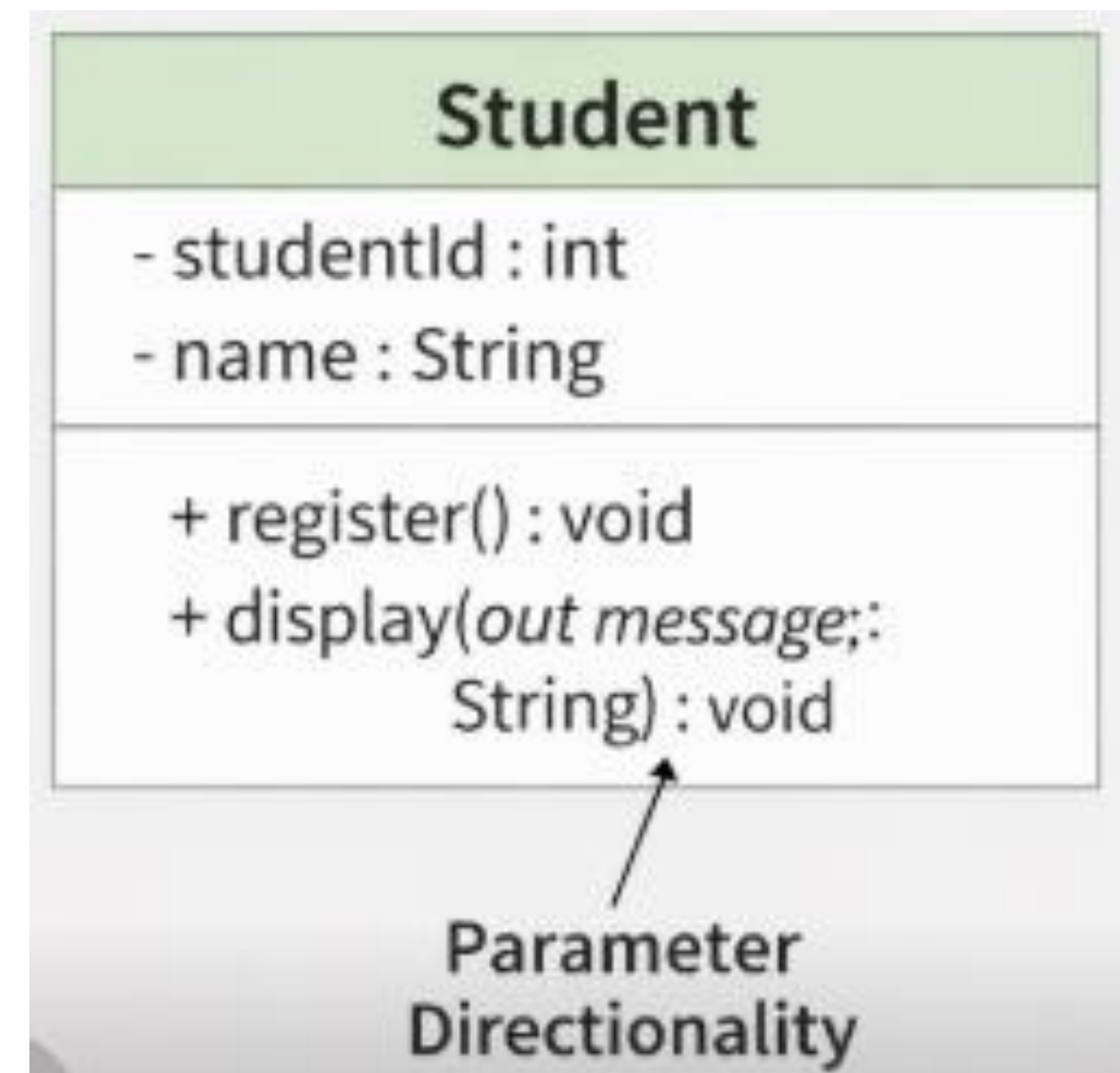
▶ UML Class Notation Cont..

▶ Parameter Directionality

Parameter directionality shows the **flow of information** between classes through method parameters.

It helps to identify whether a parameter is:

- Input
- Output
- Both input and output



Class Diagram cont..

▶ UML Class Notation Cont..

▶ Parameter Directionality

Types of Parameter Directionality

1. In (Input)

→ Data is sent from the caller to the method. Used only for input purposes.

```
calculateTotal(in price : double)
```

2. Out (Output)

→ Data is returned from the method back to the caller. Used for output purposes.

```
getResult(out grade : String)
```

3. InOut (Input and Output)

→ Parameter is sent to the method and also returned back after modification. Used for both input and output.

```
updateBalance(inout amount : double)
```

Class Diagram cont..

► Relationships between classes

In UML class diagrams, relationships describe how classes are connected and interact with each other within a system.

1. Association

- Represents a bi-directional relationship between two classes
- Shows that objects of one class are connected to objects of another class.
- Usually represented using a solid line.



Association

Ex: A student enrolls in a course.

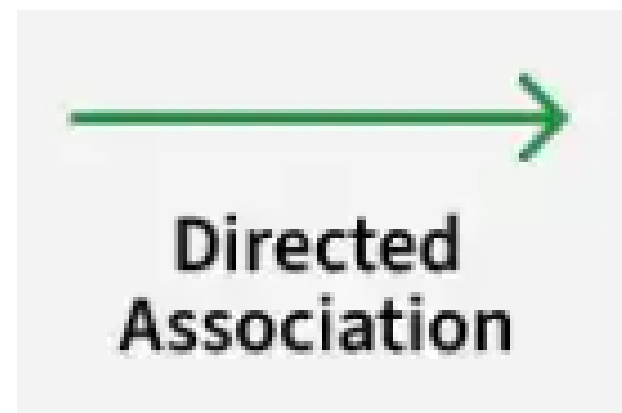


Class Diagram cont..

► Relationships between classes

2. Directed Association

- A relationship where one class uses or knows another class in a specific direction.
- one class is associated with another in a specific way.
- Represented using a solid line with an arrow.



Ex: A customer places an order.



Class Diagram cont..

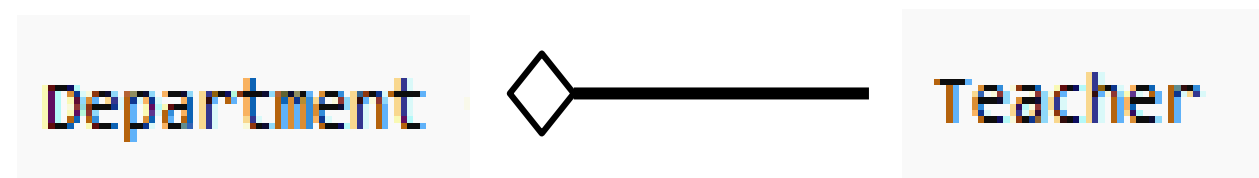
► Relationships between classes

3. Aggregation

- Represents a **whole-part relationship**.
- The child object can exist independently of the parent.
- Represented using a **hollow diamond**. (diamond shape on the side of the whole class)



Ex: A department has teachers, but teachers can exist without the department.



Class Diagram cont..

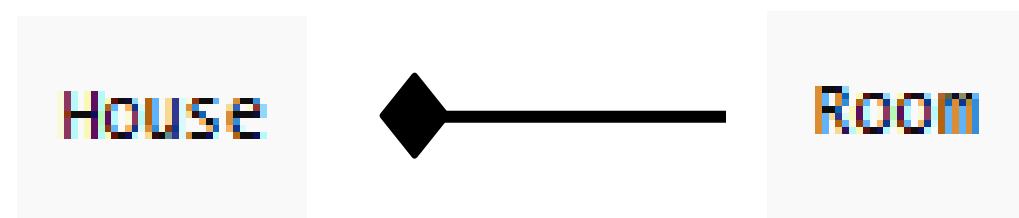
► Relationships between classes

4. Composition

- A stronger form of aggregation.
- The child object cannot exist independently of the parent.
- Represented using a filled diamond. (diamond shape on the side of the whole class.)



Ex: Rooms cannot exist without the house.

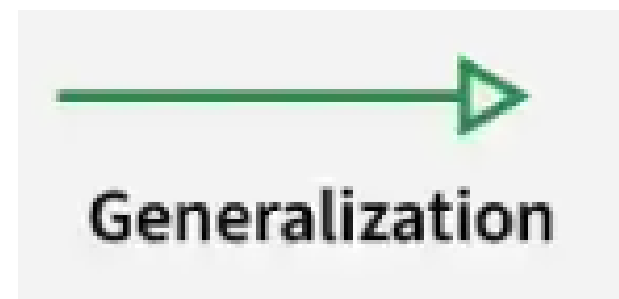


Class Diagram cont..

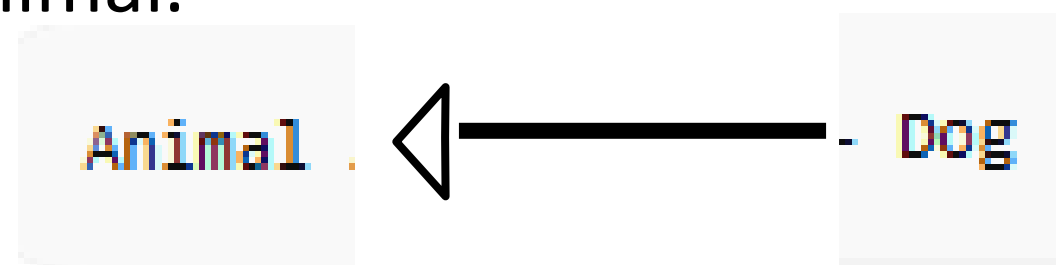
► Relationships between classes

5. Generalization (Inheritance)

- Represents an “is-a” relationship.
- One class inherits properties and methods from another class.
- Represented using a solid line with a hollow arrow. (hollow arrowhead pointing from the subclass to the superclass.)



Ex: A dog is an animal.

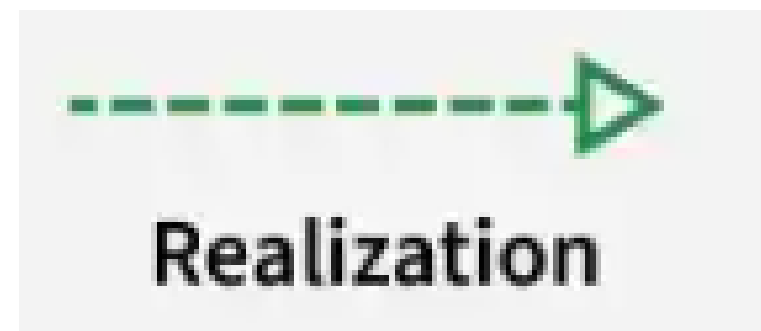


Class Diagram cont..

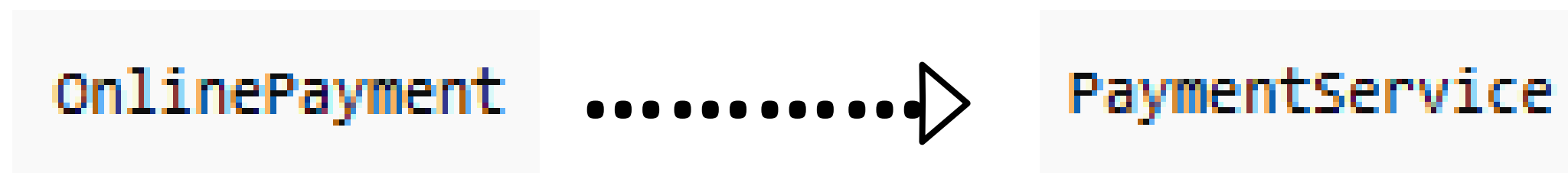
► Relationships between classes

6. Realization (Interface Implementation)

- Shows that a class implements an interface.
- Represented using a dashed line with a hollow arrow. (pointing from the implementing class to the interface.)



Ex: OnlinePayment implements PaymentService interface..

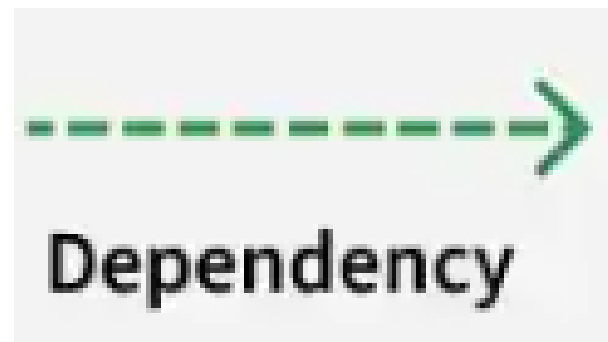


Class Diagram cont..

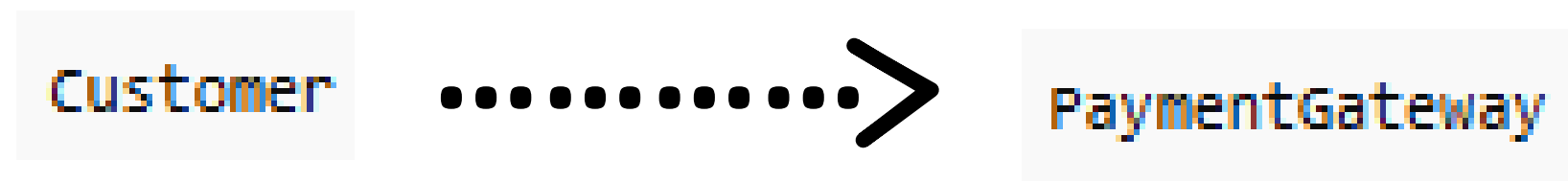
► Relationships between classes

7. Dependency Relationship

- One class temporarily depends on another class.
- Represents a weak relationship.
- Usually represented using a dashed arrow.



Ex: Customer uses PaymentGateway for payment processing.



Class Diagram cont..

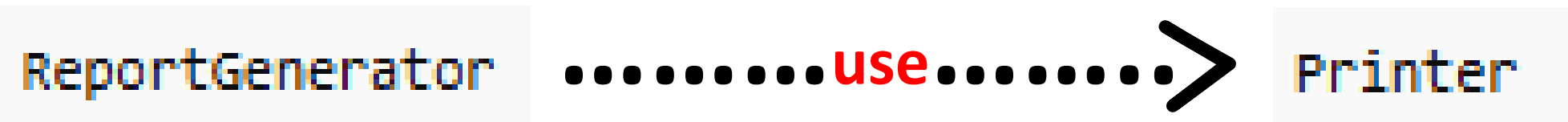
► Relationships between classes

8. Usage (Dependency) Relationship

- One class uses the services of another class.
- The client class depends on the supplier class.
- Represented using a dashed arrow.



Ex: ReportGenerator uses Printer to print reports.



Class Diagram cont..

- ▶ **Cardinality ratio/ Multiplicity Ratios**
How many objects of one class can be associated with objects of another class.

Common Cardinality / Multiplicity Ratios

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one
* or 0..*	Many
1..*	One or many
m..n	Specific range

Class Diagram cont..

► Cardinality ratio/ Multiplicity Ratios Examples

Ex 01: One-to-One (1 : 1)

One person has one passport.

One passport belongs to one person.

Person 1 ----- 1 Passport

Ex02 : One-to-Many (1 : *)

One department can have many employees.

Each employee belongs to one department.

Department 1 ----- * Employee

Ex03 : Many-to-Many (* : *)

One student can enroll in many courses.

One course can have many students.

Student * ----- * Course

Class Diagram cont..

► Cardinality ratio/ Multiplicity Ratios Examples

Ex 04: One-or-Many to Many (1..* : *)

A teacher must teach at least one subject and may teach many subjects.

A subject can be taught by many teachers.

```
Teacher 1..* ----- * Subject
```

Ex05 : Zero-or-One (0..1)

An employee may or may not have a parking slot.

Each parking slot belongs to one employee..

```
Employee 0..1 ----- 1 ParkingSlot
```

Ex06 : Zero-or-Many (0..*)

A customer can place zero or many orders.

```
Customer 1 ----- 0..* Order
```


Class Diagram cont..

► Applications of UML Class Diagram

UML Class Diagrams are widely used in software development to represent system structure and object-oriented designs.

- Used in software design and system architecture planning.
- Helps in database modeling and representing data structures.
- Useful for system documentation and maintenance.
- Widely used in object-oriented analysis and design.
- Commonly used in enterprise and large-scale application development.

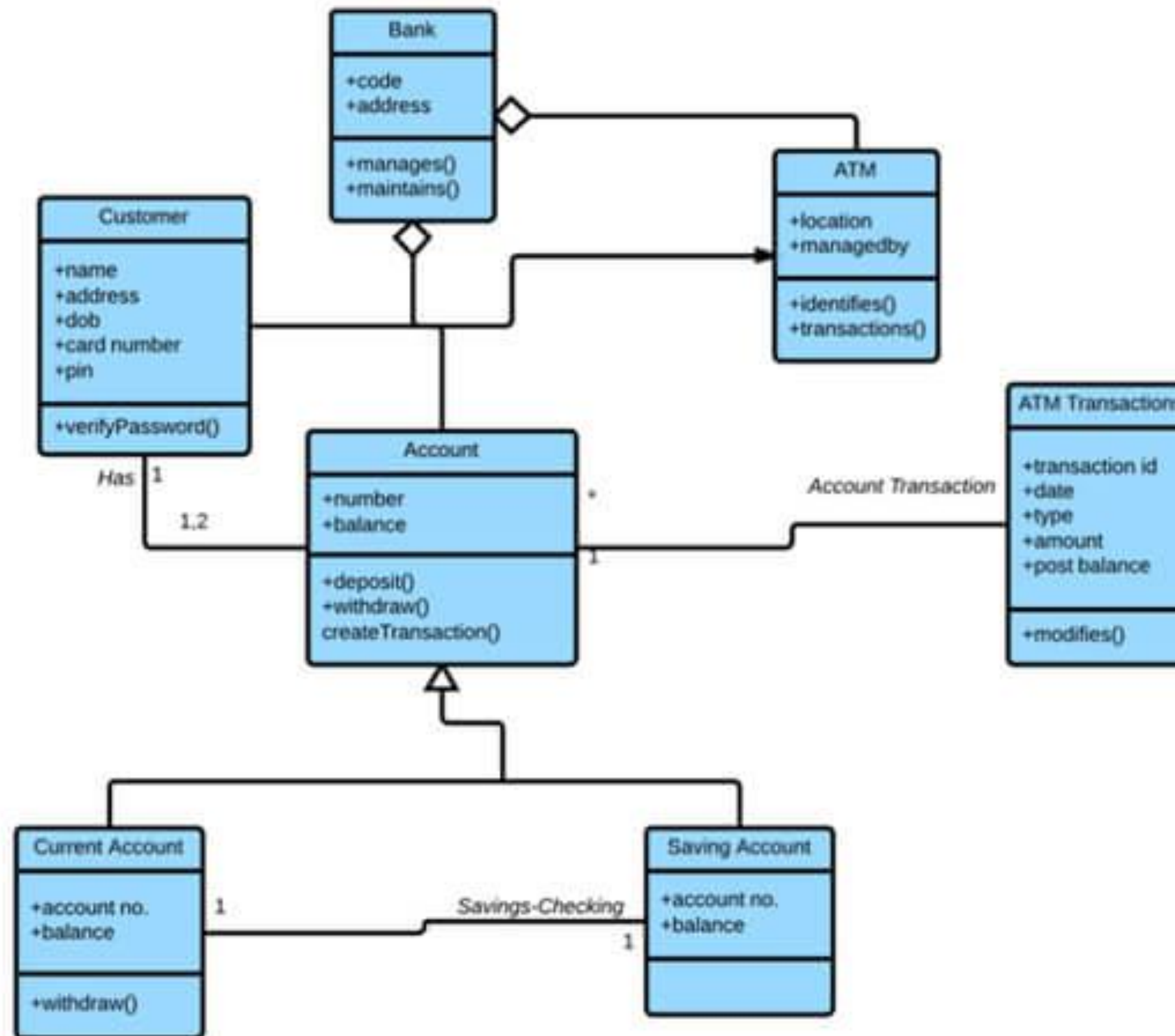
Class Diagram cont..



Advantages of UML Class Diagram

- Helps in faster and more efficient system design and analysis.
- Improves understanding of object-oriented programming (OOP) concepts.
- Enhances communication between developers, designers, and stakeholders.
- Supports forward and reverse engineering during software development.
- Provides proper documentation of the system structure and components.

Class Diagram cont..



Use Case Diagram

- ▶ A **Use Case Diagram** is a UML behavioral diagram used to represent the **functional requirements** of a system.
It shows:
 - What the system does
 - Who interacts with the system
 - How users interact with the system
- ▶ Use case diagrams help developers and stakeholders understand the system from the **user's perspective**.
- ▶ **Main Purpose of a Use Case Diagram**
 - Identify system functionalities
 - Show interactions between users and the system
 - Capture functional requirements
 - Simplify system analysis and communication

Use Case Diagram cont..

► Main Components of a Use Case Diagram

1. Actor

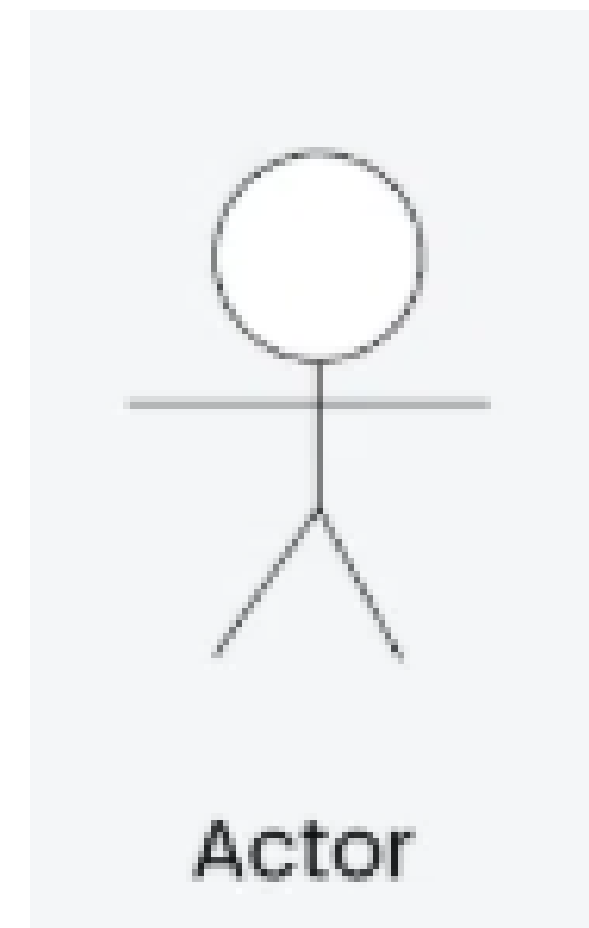
An Actor represents a user or external system that interacts with the system.

Symbols

- Usually represented using a **stick figure**

Examples

- Student
- Customer
- Admin
- Payment Gateway



Use Case Diagram cont..

► Main Components of a Use Case Diagram

2. Use Case

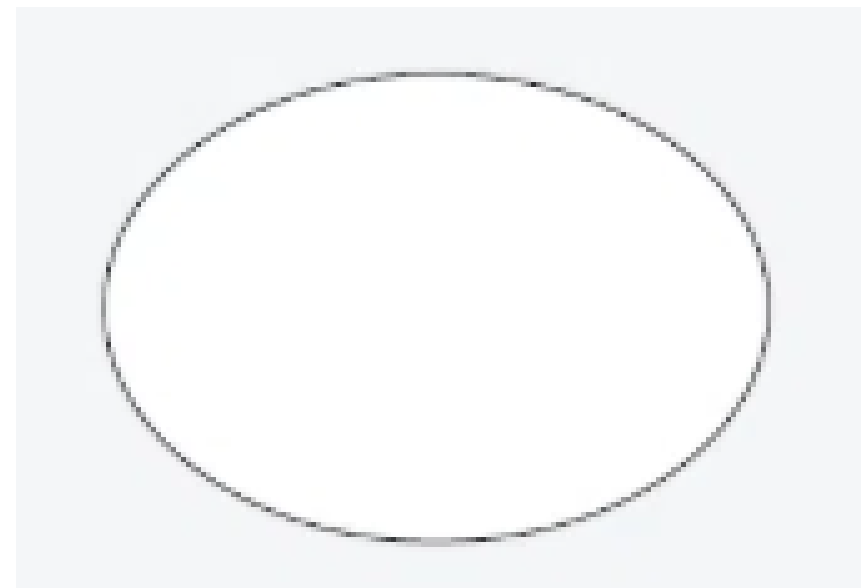
A Use Case represents a function or service provided by the system.

Symbols

- Represented using an **oval shape**

Examples

- Student
- Customer
- Admin
- Payment Gateway



Use Case Diagram cont..

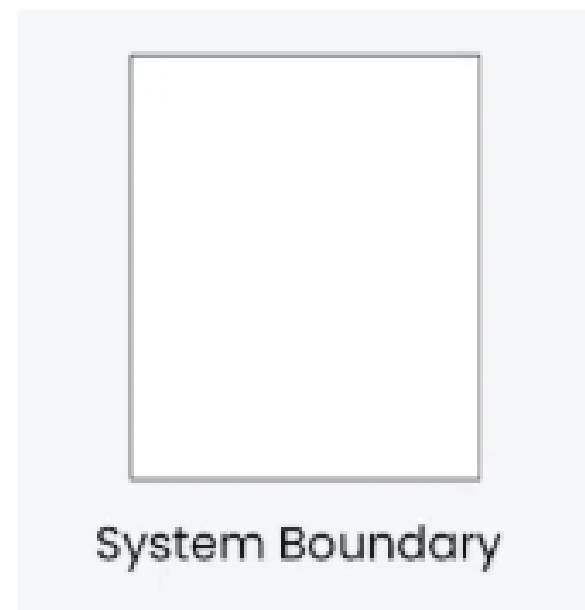
▶ Main Components of a Use Case Diagram

3. System Boundary

The System Boundary defines the scope of the system.

Symbols

- Represented using a rectangle
- All use cases are placed inside the boundary.



Use Case Diagram cont..

► Relationships in Use Case Diagrams

1. Association Relationship

Represents interaction between an actor and a use case..

Symbol

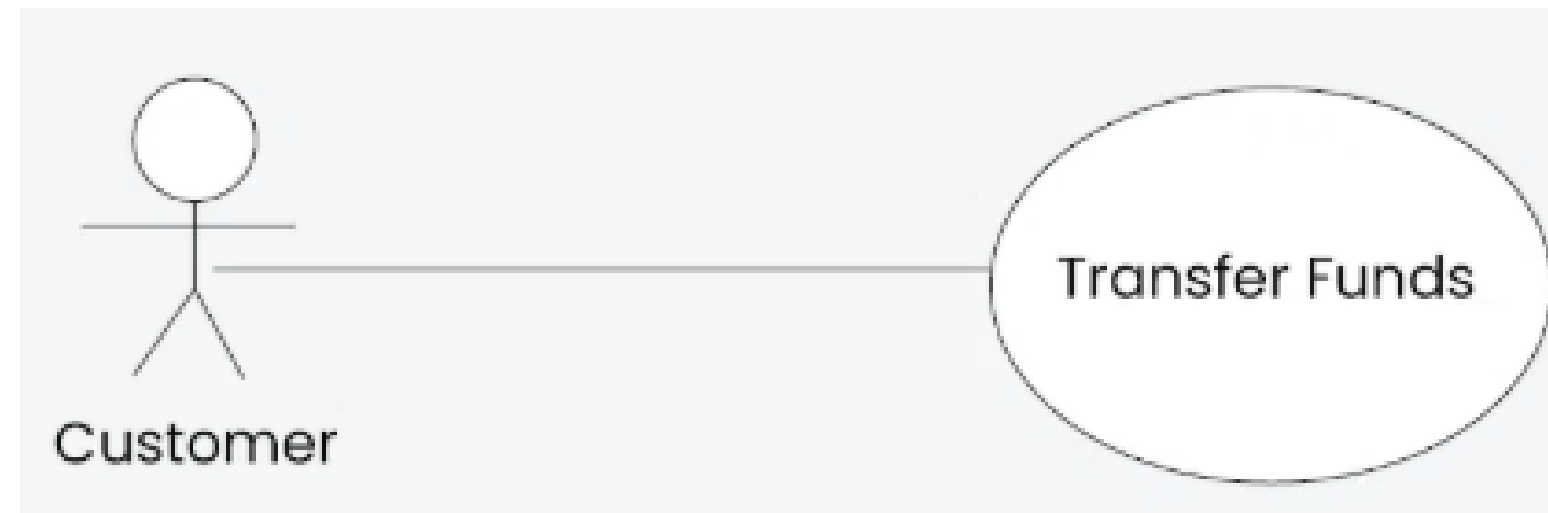
- Solid line

Example:

Online Banking System

Actor: Customer

Use Case: Transfer Funds



Use Case Diagram cont..

► Relationships in Use Case Diagrams

2. Include Relationship

Used when one use case always includes another use case.

Symbol

- Dashed arrow with <<include>>

Example:

Login always includes user validation.

Use Case: Login , Validate User



Use Case Diagram cont..

► Relationships in Use Case Diagrams

3. Extend Relationship

Used when additional functionality is added under certain conditions.

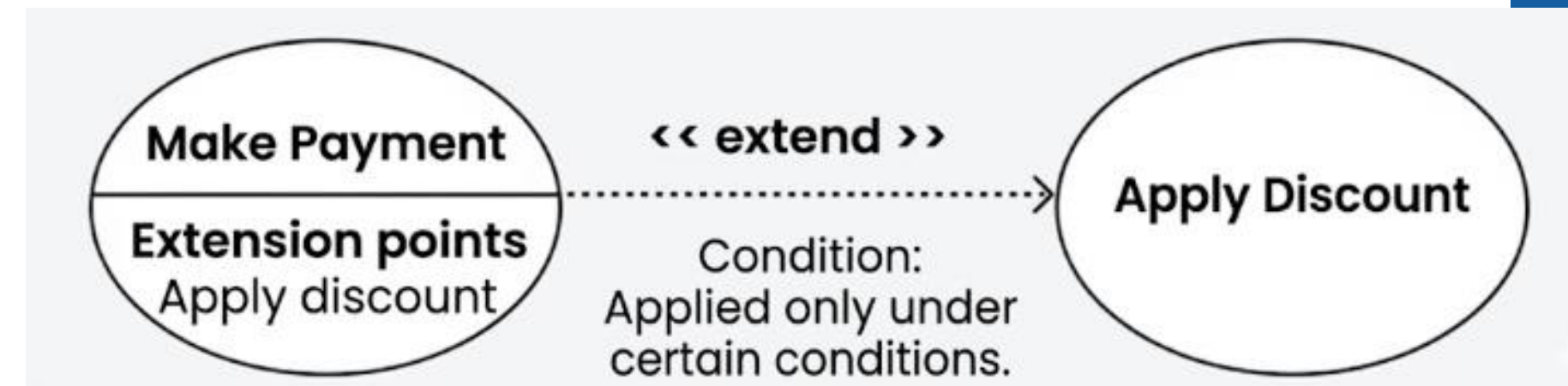
Symbol

- Dashed arrow with <<extend>>

Example:

Discount is applied only under certain conditions.

Use Cases: Make Payment, Apply Discount



Use Case Diagram cont..

► Relationships in Use Case Diagrams

4. Generalization Relationship

- Represents inheritance between actors or use cases.
- Establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another
- Arrow pointing from the specialized use case to the general use case.

Symbol

- Solid line with hollow triangle



Example:

Admin is a specialized type of User.

Actor: Admin , User.

Use Case Diagram cont..

Advantages of Use Case Diagrams

- Easy to understand
- Good for requirement gathering
- Improves communication with stakeholders
- Shows system functionality clearly
- Useful before coding begins

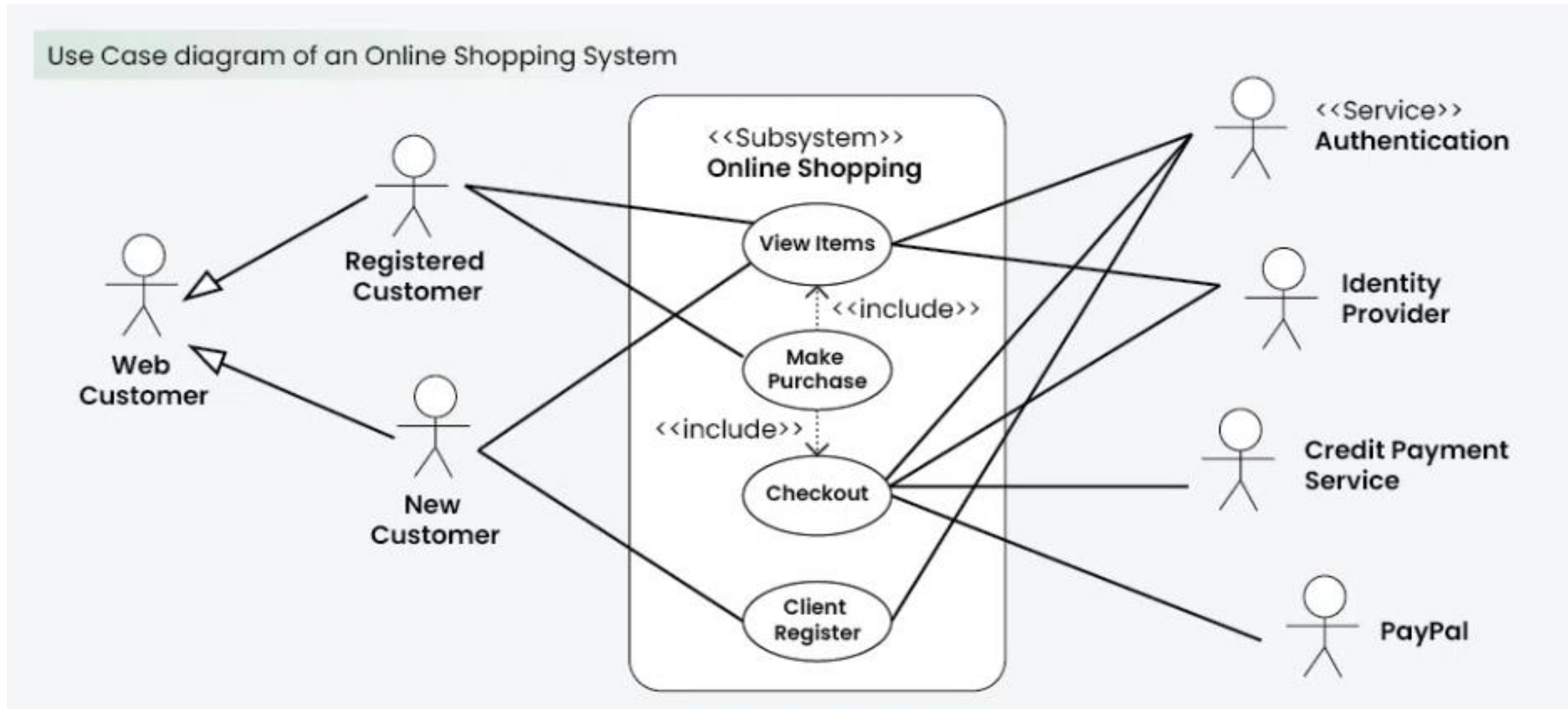
Use Case Diagram cont..

Limitations of Use Case Diagrams

- Does not show internal system logic
- Cannot describe detailed workflows
- Large systems can become complex

Use Case Diagram cont..

▶ Sample Use Case Diagram



Activity Diagram

What is an Activity Diagram

An Activity Diagram is a behavioral UML diagram used to represent the flow of activities, actions, and decisions within a system or business process.

It shows:

- Sequence of activities
- Decision-making points
- Parallel processing
- Start and end points of workflows

Purpose:

To visualize how a process operates from beginning to end.

Activity Diagram Cont...



Why Use Activity Diagrams?

- Modeling Workflows or Processes
- Concurrent or Parallel Processing
- Understanding the Dynamic Behavior
- Clarifying Complex Logic
- System Design and Analysis
- Describing Use Cases

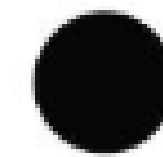
Activity Diagram Cont...



Basic Notations

1. Initial Node

- Represents where the workflow begins.
- Only one initial node is commonly used
- use a black filled circle



Initial State

Example : ATM Cash Withdrawal



Activity Diagram Cont...



Basic Notations

2. Action or Activity State

- Activities represent actions performed by users, systems, or processes.
- represent an activity using a rectangle with rounded corners



Activity State

Example : Enter Username
Validate User
Generate Report
Process Payment



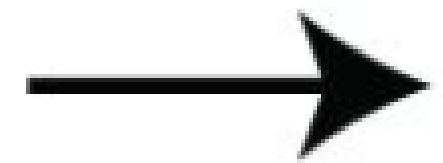
Activity Diagram Cont...



Basic Notations

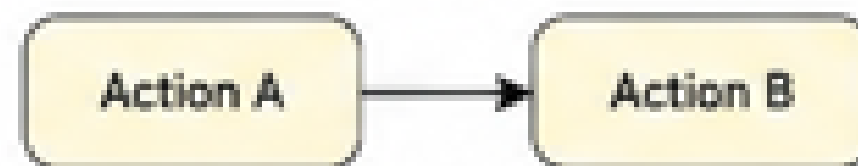
3. Action Flow or Control flows

- Control flow arrows indicate the order in which activities occur.
- activity state can have multiple incoming and outgoing action flows.
- Use a line with an arrow head to depict a Control Flow



Control Flow

Example : Action B occur after Action A

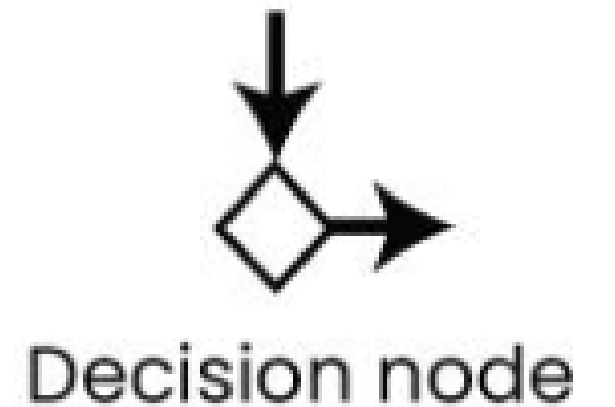


Activity Diagram Cont...

► Basic Notations

4. Decision node and Branching

- Used when a condition determines different paths. (need to make a decision)
- The outgoing arrows from the decision node can be labelled with conditions or guard expressions.
- It always includes two or more output arrows.



Example : User Login

Valid Credentials?

Yes → Access System

No → Display Error Message



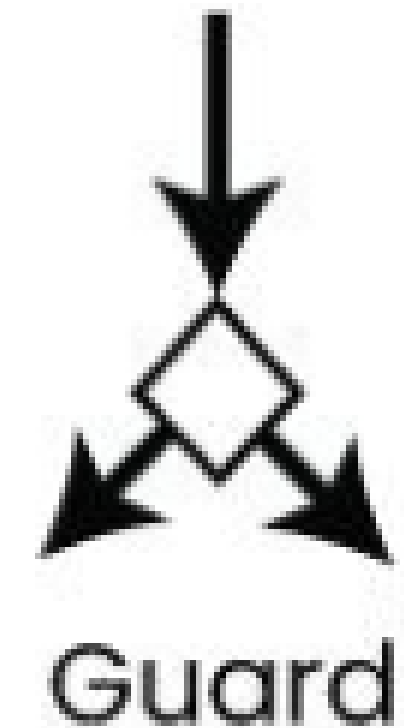
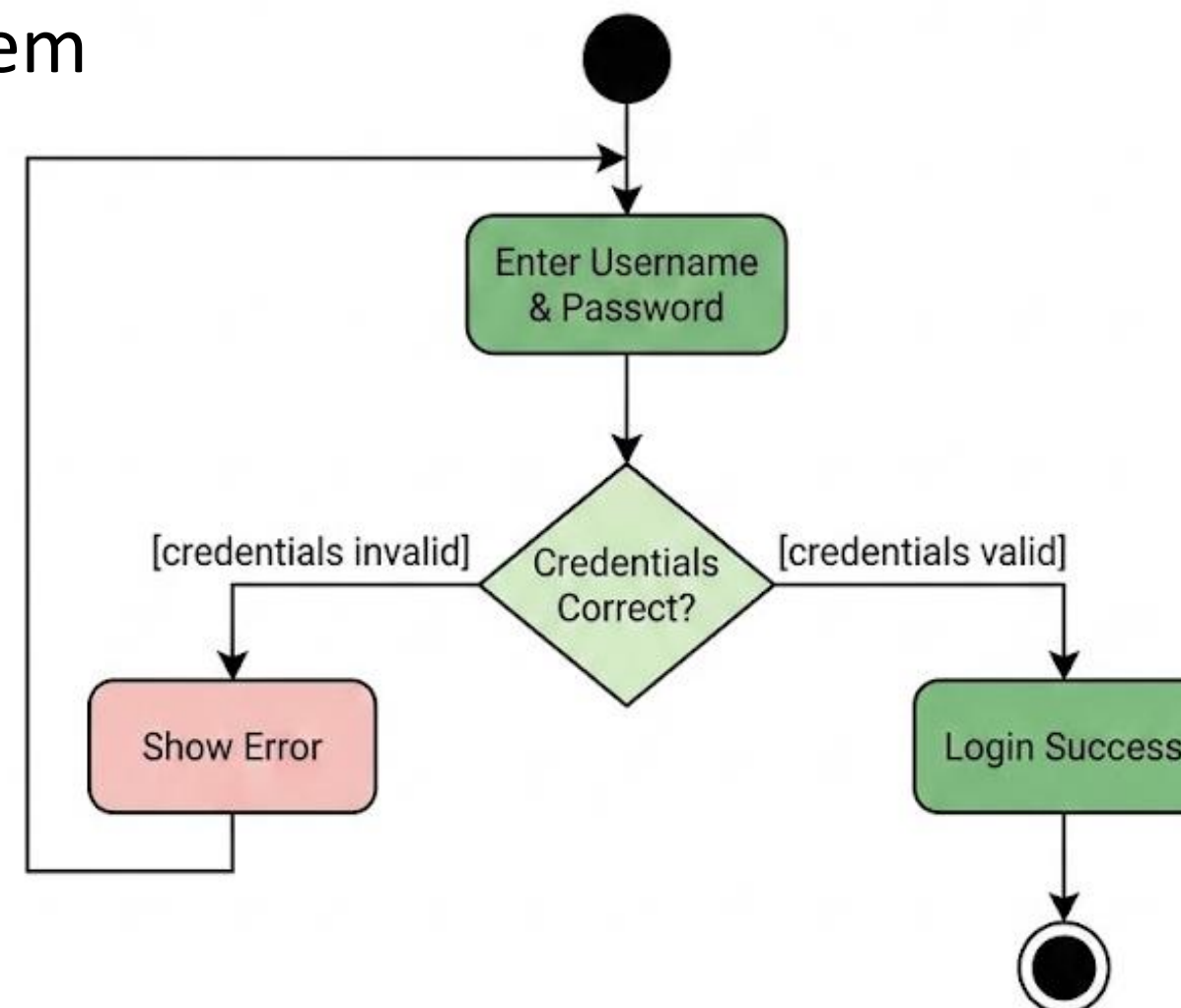
Activity Diagram Cont...

► Basic Notations

5. Guard

- a statement written next to a decision node on an arrow
- **Most of the time use square brackets.**

Example :User Logging System

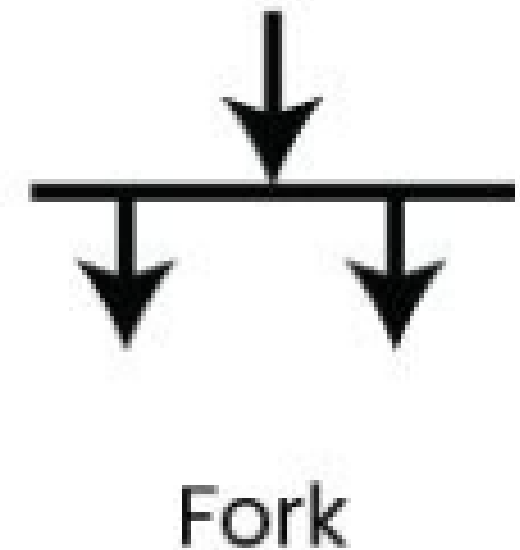


Activity Diagram Cont...

▶ Basic Notations

6. Fork

- Used to split one flow into multiple parallel activities.(concurrent activities)
- no decision is made before splitting the activity into two parts.
- Both parts need to be executed in case of a fork statement



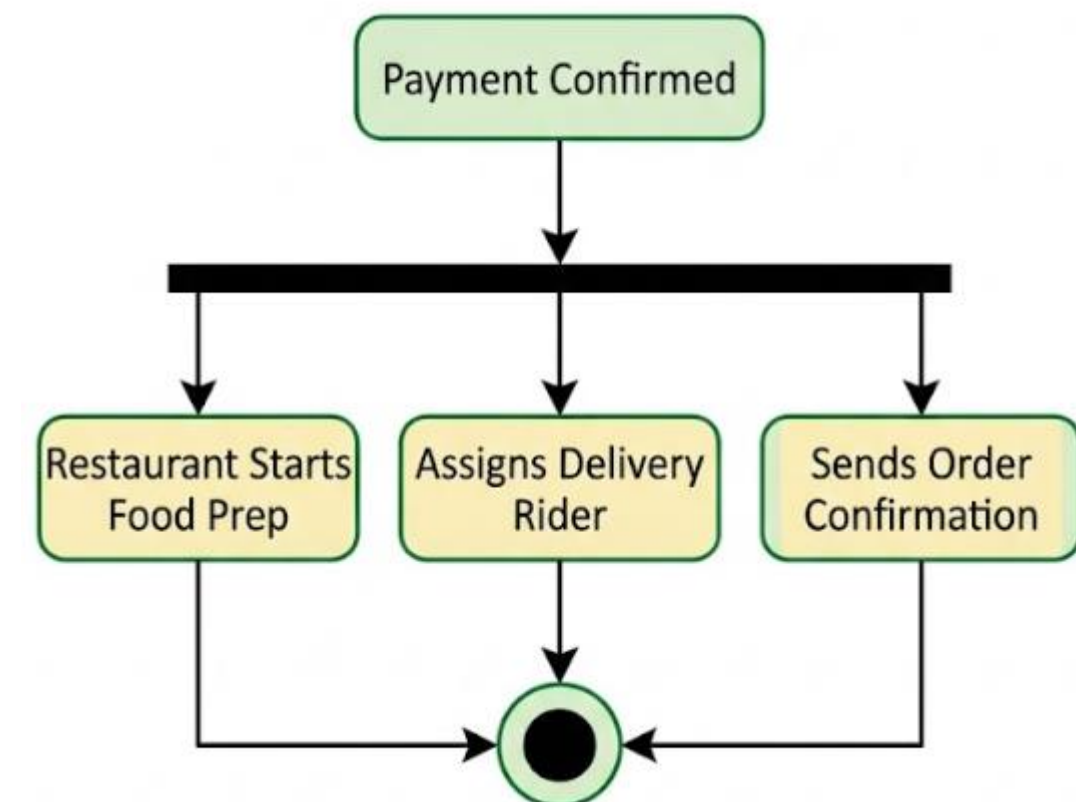
Example : Imagine you order food from an app like Uber Eats or any food delivery system.

Step-by-step flow:

- 1.Customer places an order
- 2.Payment is confirmed
3. The system **forks (splits into parallel tasks)**:

After fork (parallel activities happen at the same time):

- Restaurant starts preparing the food
- Delivery system assigns a delivery rider
- Customer receives order confirmation notification



These three activities happen simultaneously, not one after another.

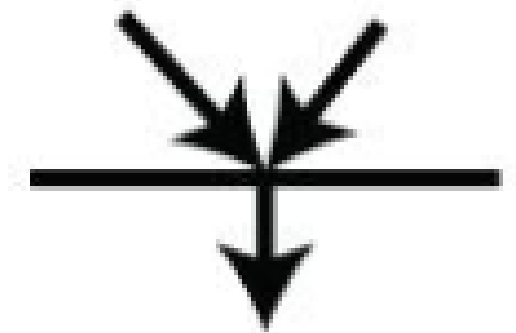
Activity Diagram Cont...



Basic Notations

7. Join

- Combines parallel activities before proceeding.
- For join notations we have two or more incoming edges and one outgoing edge.



Join

Activity Diagram Cont...

▶ Basic Notations

7. Join

Example :In food ordering system, after the payment verification bellow activities are performed parallel

Restaurant prepares the food

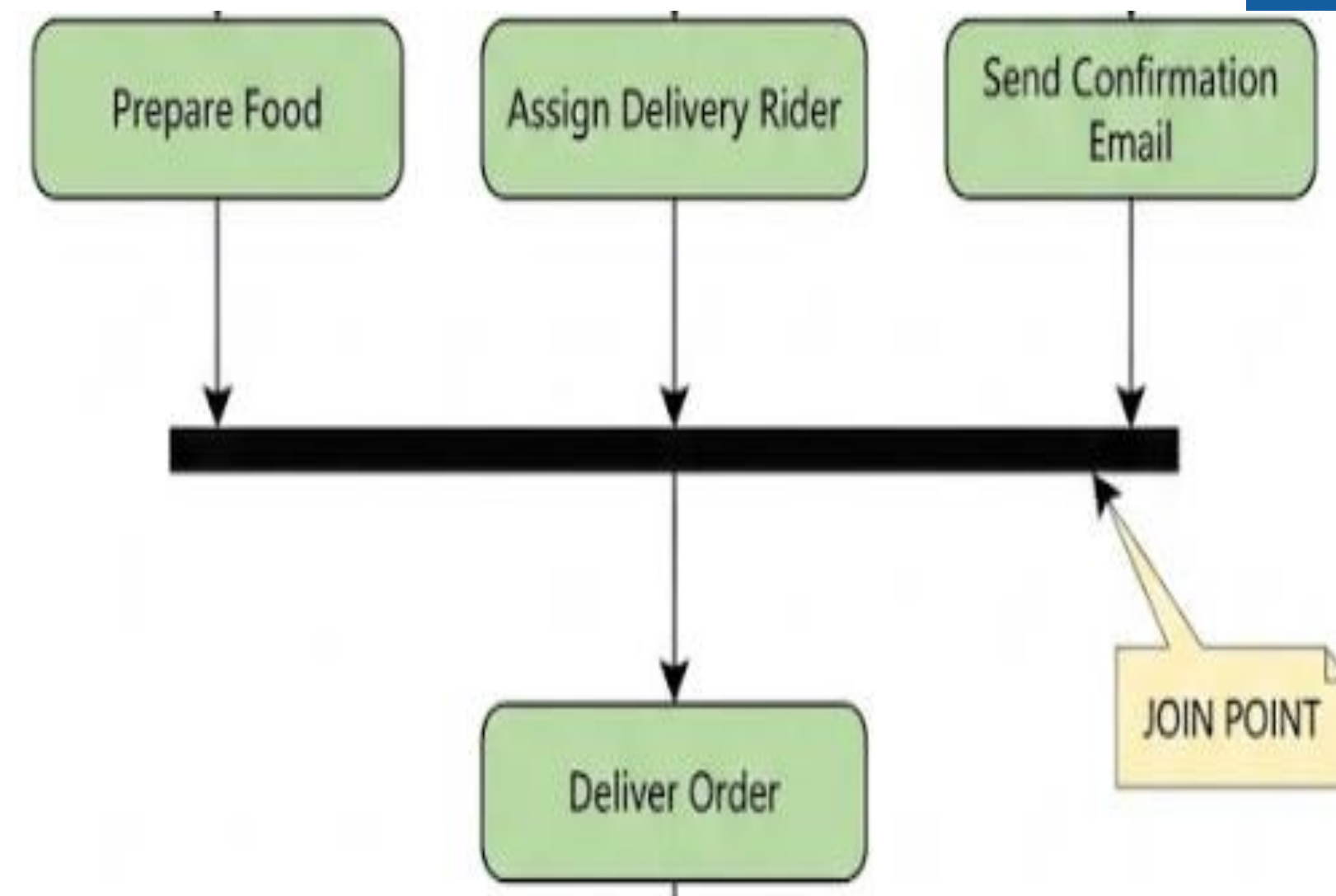
Delivery rider is assigned

Customer gets confirmation notification

join happens when all required tasks are completed:

Final step:

Delivery starts (rider goes to customer location)

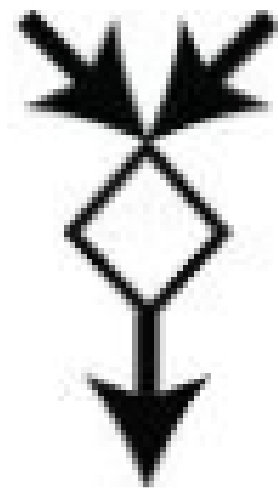


Activity Diagram Cont...

▶ Basic Notations

8. Merge or Merge Event

- when activities which are not being executed concurrently have to be merged
- We can merge two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen.



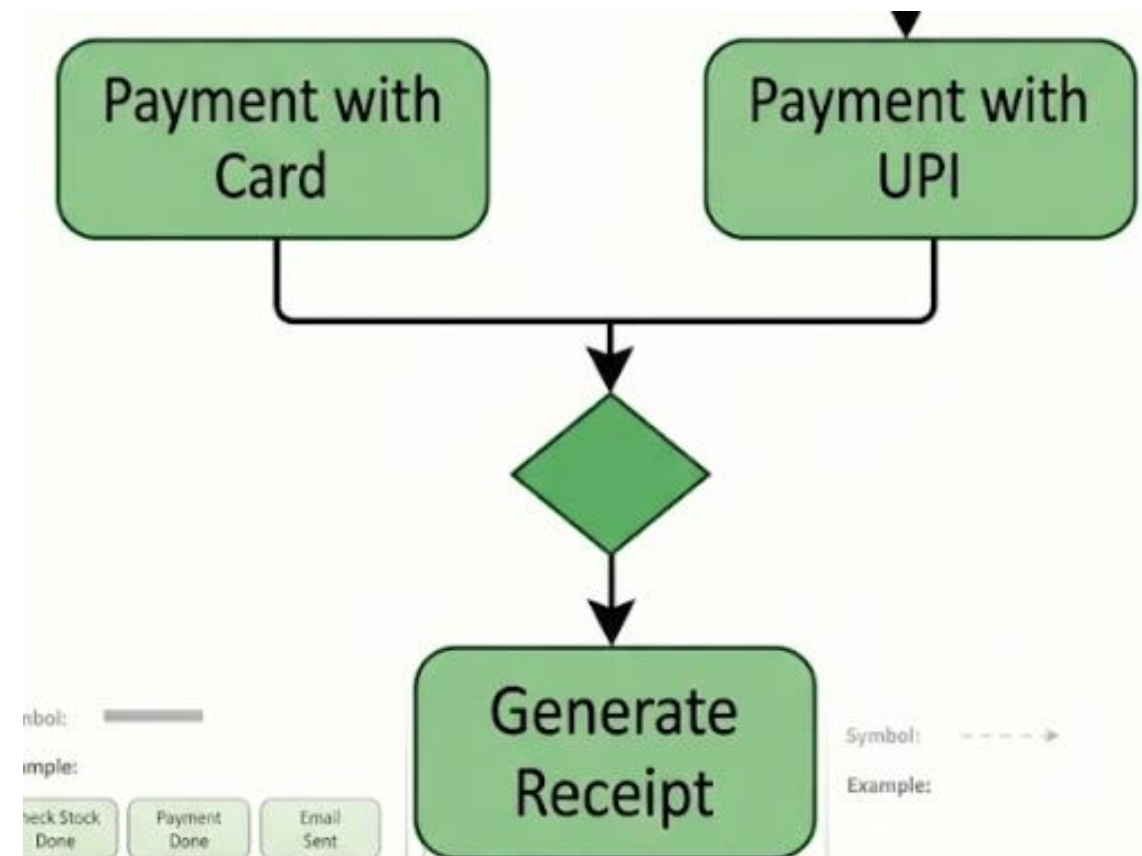
Merge

Activity Diagram Cont...

► Basic Notations

8. Merge or Merge Event

Example : you can do the payments using card or UPI.. After that it generate Receipt



Activity Diagram Cont...



Basic Notations

9. Swimlanes

- Swimlanes divide activities according to responsible actors or departments.
- Swimlanes group related activities into one column or one row.
- can be vertical and horizontal.
- not mandatory to use
- It's similar to creating a function in a program

Benefits

- Shows responsibility clearly.
- Improves process understanding.
- Useful for business process modeling.



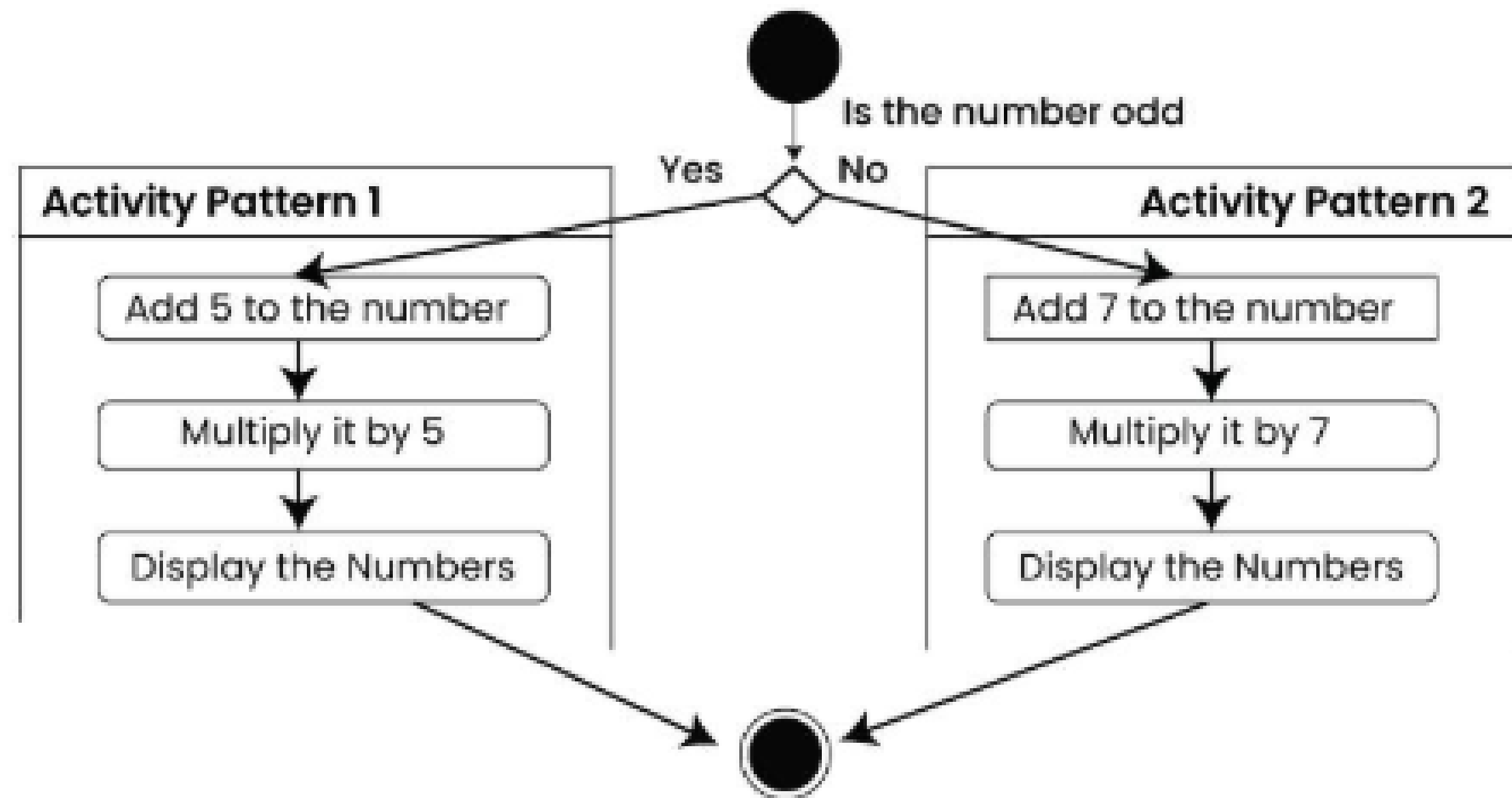
Swimlane

Activity Diagram Cont...

► Basic Notations

9. Swimlanes

Example :



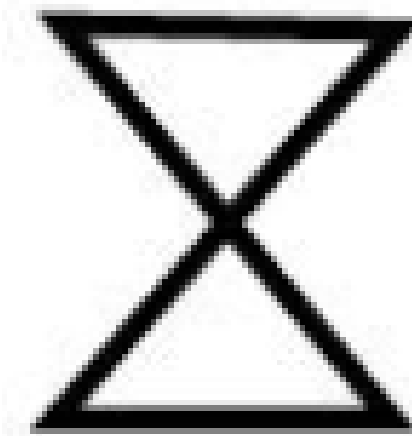
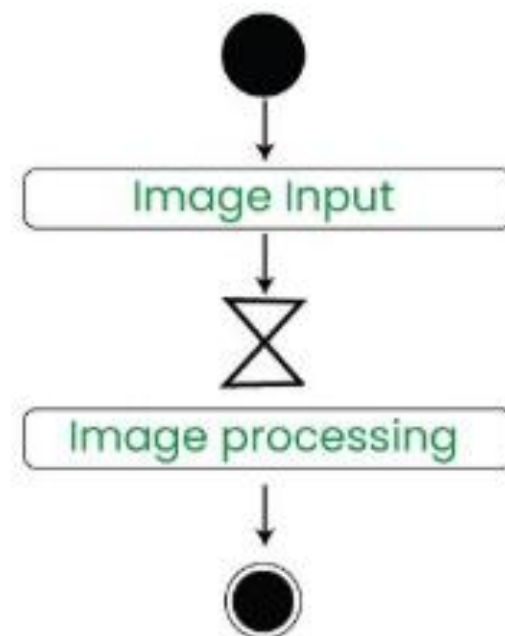
Activity Diagram Cont...

▶ Basic Notations

10. Time Event

- This refers to an event that stops the flow for a time; an hourglass depicts it. We can have a scenario where an event takes some time to completed.

Example : *Let us assume that the processing of an image takes a lot of time. Then it can be represented as shown below.*



Time Event

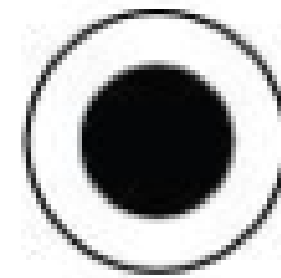
Activity Diagram Cont...



Basic Notations

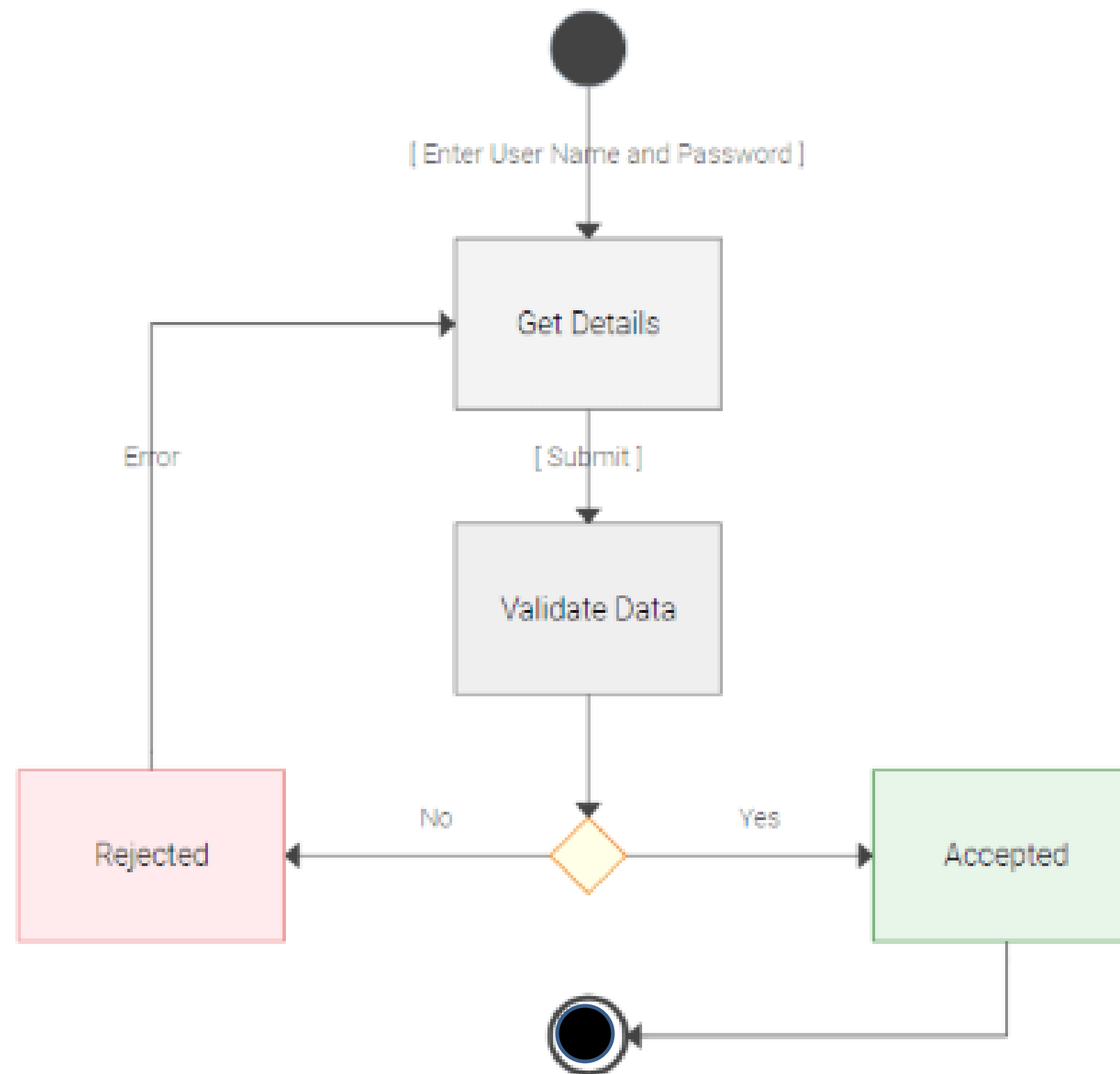
11. Final State or End State

- Represents completion of the process.
- Indicates no further activities will occur..



Final State

Activity Diagram Cont...



Activity Diagram Cont...



Best Practices

- Keep diagrams simple and readable.
- Use meaningful activity names.
- Avoid excessive branching.
- Clearly label decision conditions.
- Use swimlanes when multiple actors are involved.
- Maintain a logical flow from top to bottom.

Activity Diagram Cont...



Advantages

- Easy to understand complex processes
- Shows step-by-step workflow clearly
- Supports parallel processing (fork & join)
- Helps identify decision points
- Improves communication
- Useful for system and business modeling
- Helps in system design and documentation

Activity Diagram Cont...



Limitations of Activity Diagrams

- Can become complex for large systems
- Not suitable for showing detailed object interactions
- Difficult to maintain for frequently changing processes
- Large diagrams may reduce readability

State Diagrams

A **State Diagram (State Machine Diagram)** shows how an object behaves by describing its **different states and transitions between them** based on events.

It models the lifecycle of an object by showing different states and the transitions between them.

It focuses on:

- States of an object
- Events that change states
- Conditions (guards)
- Lifecycle of an object

State Diagrams Cont..

Purpose of State Diagrams

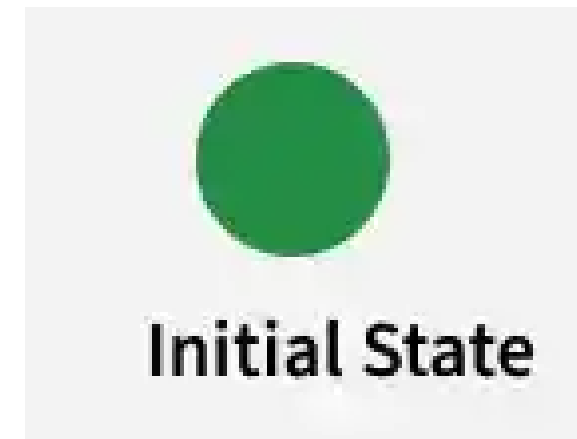
- To model the lifecycle of an object
- To understand how systems react to events
- To track state changes over time
- To improve system behavior design
- To represent real-world dynamic systems

State Diagrams Cont..

Basic Components and Notations

1. Initial state

We use a black filled circle represent the initial state of a System or a Class.



2. Transition

- use a solid arrow to represent the transition or change of control from one state to another.
- The arrow is labelled with the event which causes the change in state..



State Diagrams Cont..

Basic Components and Notations

3. State

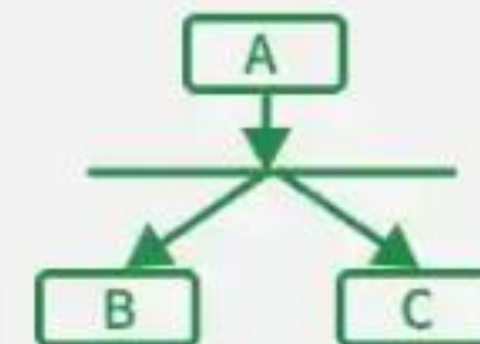
- use a rounded rectangle to represent a state.
- A state represents the conditions or circumstances of an object of a class at an instant of time.



State

4. Fork

- use a rounded solid rectangular bar to represent a Fork notation **with incoming arrow from the parent state and outgoing arrows towards the newly created states.**
- We use the fork notation to represent a state splitting into two or more concurrent states.



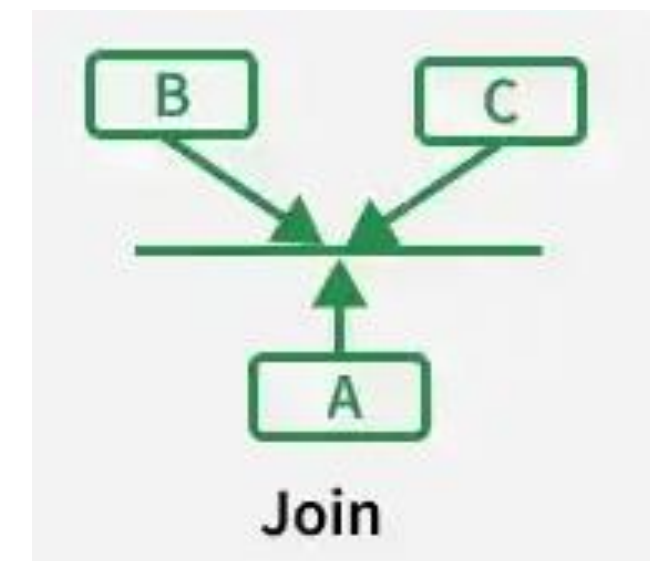
Fork

State Diagrams Cont..

Basic Components and Notations

5. Join

- use a rounded solid rectangular bar to represent a Join notation with **incoming arrows from the joining states** and **outgoing arrow towards the common goal state**.
- use the join notation when two or more states concurrently converge into one on the occurrence of an event or events.



6. Self transition

- use a solid arrow pointing back to the state itself to represent a self transition.
- There might be scenarios when the state of the object does not change upon the occurrence of an event.

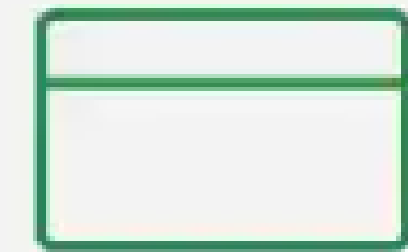


State Diagrams Cont..

Basic Components and Notations

7. Composite state

- use a rounded rectangle to represent a composite state
- represent a state with internal activities



Composite State

8. Final State

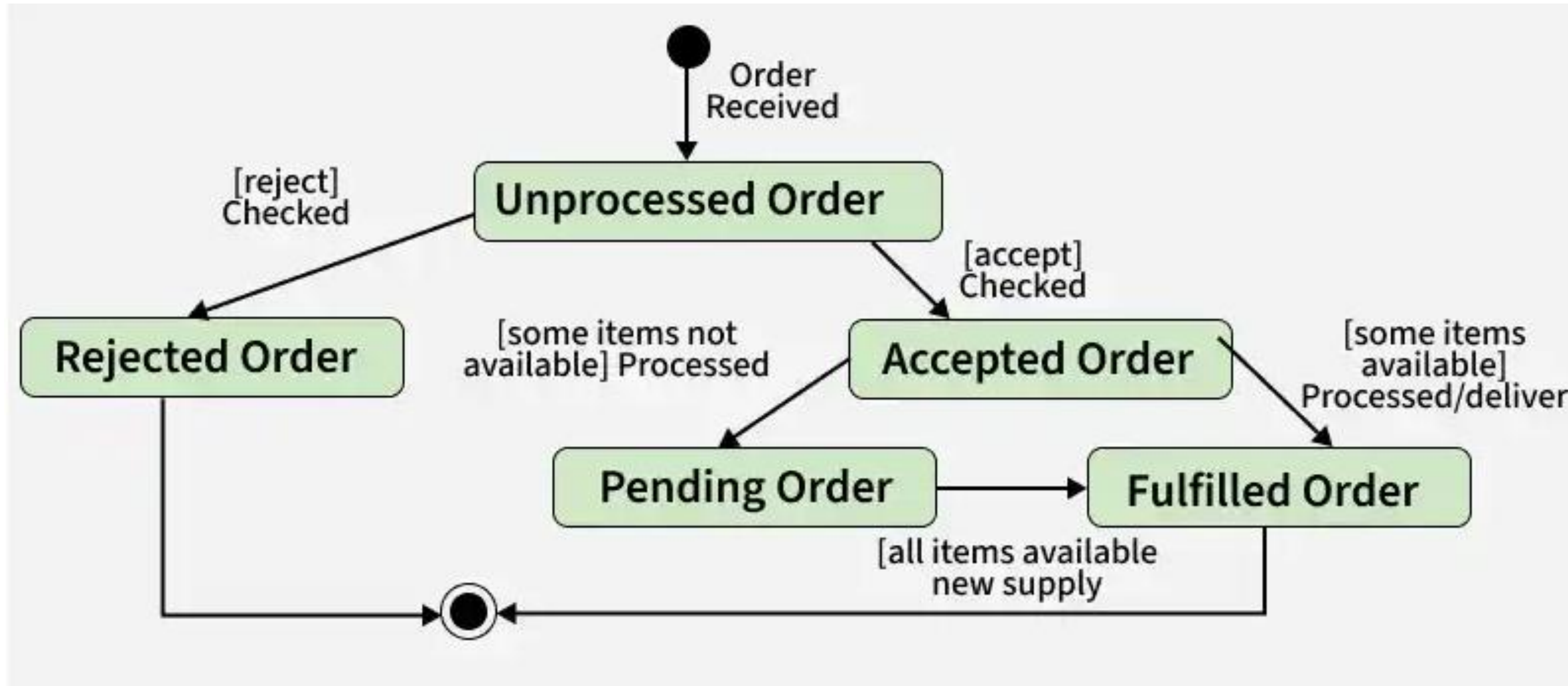
- We use a filled circle within a circle notation to represent the final state in a state machine diagram.



Final State

State Diagrams Cont..

State Machine for an Online Order



State Diagrams Cont..

Draw a State Machine diagram in UML

Step 1: Identify the System

Step 2: Identify Initial and Final States

Step 3: Identify Possible States

Step 4: Label Triggering Events

Step 5: Draw the Diagram with appropriate notations

State Diagrams Cont..

Advantages of State Diagrams

1. Easy to understand system behavior
2. Clearly shows object lifecycle
3. Helps in event-driven systems
4. Useful for complex state management
5. Improves system design accuracy

State Diagrams Cont..

Limitations of State Diagrams

1. Not suitable for simple systems
2. Becomes complex with many states
3. Focuses only on one object at a time
4. Hard to maintain in large systems

Summary

- ▶ What is UML
- ▶ Importance of UML
- ▶ UML Tools (brief intro)
- ▶ Types of UML
 - ▶ Class Diagram
 - ▶ Use Case Diagram
 - ▶ Activity Diagram
 - ▶ State Diagram

The background is a blurred office scene with people working at desks, overlaid with a semi-transparent blue filter. A white rectangular box is centered on the image, containing the text "ANY QUESTIONS ?" in a bold, white, sans-serif font. The word "ANY" is on the top line, and "QUESTIONS ?" is on the bottom line.

**ANY
QUESTIONS ?**



THANK YOU!

